Experiment 13

Implement your own ping program

Aim

To implement a ping program using ICMP Echo Requests and Replies.

Requirements

Python 3.10+, IPv4 network and raw ICMP socket privileges.

Procedure / algorithm

1. Resolve the hostname to IPv4 and open an AF_INET SOCK_RAW IPPROTO_ICMP socket.

2. Build Echo Request type 8/code 0 with identifier, changing sequence and payload. Calculate the one's-complement checksum with the checksum field initially zero.

3. Send the packet, record monotonic time and wait until the timeout deadline.

4. Use IPv4 IHL to locate ICMP; validate total length, protocol, source, type/code, identifier, sequence and checksum. Ignore fragments and unrelated packets.

5. Accept only the matching reply with the correct payload. Measure RTT; repeat for the requested count and interval.

6. Display replies/timeouts, received/lost counts and min/average/max RTT.

Complete source / configuration: ping.py

```
"""IPv4 ICMP Echo using raw sockets; no system ping wrapper."""
import argparse
import os
import socket
import struct
import time


def checksum(data):
    if len(data) % 2:
        data += b"\0"
    total = sum(struct.unpack(f"!{len(data)//2}H", data))
    while total >> 16:
        total = (total & 0xffff) + (total >> 16)
    return (~total) & 0xffff


def request(identifier, sequence, payload=b"CS23532 CN lab"):
    header = struct.pack("!BBHHH", 8, 0, 0, identifier, sequence)
    value = checksum(header + payload)
    return struct.pack("!BBHHH", 8, 0, value, identifier, sequence) + payload


def parse_reply(packet, target, identifier, sequence):
    if len(packet) < 20 or packet[0] >> 4 != 4:
        return None
    ihl = (packet[0] & 15) * 4
    total = struct.unpack("!H", packet[2:4])[0]
    if ihl < 20 or total < ihl + 8 or len(packet) < total:
        return None
    if packet[9] != 1 or socket.inet_ntoa(packet[12:16]) != target:
        return None
    fragment = struct.unpack("!H", packet[6:8])[0]
    if fragment & 0x3fff:
        return None
    icmp = packet[ihl:total]
    kind, code, _, rid, rseq = struct.unpack("!BBHHH", icmp[:8])
    if (kind, code, rid, rseq) != (0, 0, identifier, sequence):
        return None
    if checksum(icmp) != 0:
        return None
    return {"ttl": packet[8], "bytes": len(icmp), "payload": icmp[8:]}

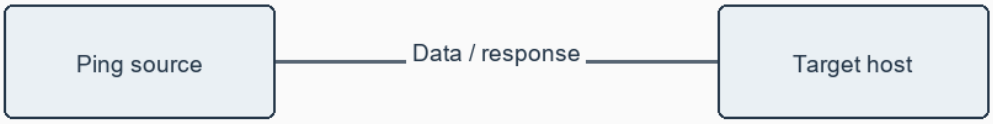

def ping(host, count=4, timeout=2, interval=1):
    if count < 1 or timeout <= 0 or interval < 0:
        raise ValueError("Count/timeout must be positive; interval nonnegative")
    target = socket.gethostbyname(host)
    identifier = os.getpid() & 0xffff
    timings = []
    payload = b"CS23532 CN lab"
    print(f"PING {host} ({target})")
    with socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP) as s:
        for sequence in range(1, count + 1):
            start = time.perf_counter()
            s.sendto(request(identifier, sequence, payload), (target, 0))
            deadline = start + timeout
            matched = False
            while time.perf_counter() < deadline:
```

```
                s.settimeout(max(.001, deadline - time.perf_counter()))
                try:
                    packet, _ = s.recvfrom(65535)
                except socket.timeout:
                    break
                reply = parse_reply(packet, target, identifier, sequence)
                if reply and reply["payload"] == payload:
                    elapsed = (time.perf_counter() - start) * 1000
                    timings.append(elapsed)
                    print(f"Reply from {target}: seq={sequence} "
                          f"bytes={reply['bytes']} TTL={reply['ttl']} "
                          f"time={elapsed:.3f} ms")
                    matched = True
                    break
            if not matched:
                print(f"Request timed out: seq={sequence}")
            if sequence < count:
                time.sleep(interval)
    loss = (count - len(timings)) * 100 / count
    print(f"Sent={count} Received={len(timings)} Loss={loss:.1f}%")
    if timings:
        print(f"RTT min/avg/max = {min(timings):.3f}/"
              f"{sum(timings)/len(timings):.3f}/{max(timings):.3f} ms")
    return timings


def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("host")
    p.add_argument("--count", type=int, default=4)
    p.add_argument("--timeout", type=float, default=2)
    p.add_argument("--interval", type=float, default=1)
    a = p.parse_args()
    try:
        ping(a.host, a.count, a.timeout, a.interval)
    except PermissionError:
        raise SystemExit("Raw ICMP requires administrator/root or CAP_NET_RAW.")


if __name__ == "__main__":
    main()
```

Diagram: icmp



Echo Request type 8 -> ; <- Echo Reply type 0.

Run

From the repository root unless a cd command is shown.

```
python Exercise_13/ping.py 127.0.0.1 --count 4
python Exercise_13/ping.py 127.0.0.1 --count 2 --interval 0.1 --timeout 1
```

Actual output / verification

```
ACTUAL RAW-SOCKET LOOPBACK RUN
PING 127.0.0.1 (127.0.0.1)
Reply from 127.0.0.1: seq=1 bytes=22 TTL=128 time=0.353 ms
Reply from 127.0.0.1: seq=2 bytes=22 TTL=128 time=0.291 ms
Sent=2 Received=2 Loss=0.0%
RTT min/avg/max = 0.291/0.322/0.353 ms

Actual offline tests: checksum known vector, request checksum,
matching replies with IP options, mismatch and truncation checks passed.
```

Expected output - not an execution record

A reachable ICMP-responsive target yields matching Echo Replies; other targets may time out. RTT and TTL vary by host/network. Run with raw-socket privileges where required.

Observations

The custom raw-socket program actually ran against 127.0.0.1 and received two matching replies. Measured times are recorded in actual.txt. Offline tests also cover the checksum known vector, IP options, mismatched identifiers and truncated replies.

Result

The custom ICMP ping program was executed on localhost and its packet/checksum logic was verified.